

BloodLink v2.0 — Software Test Plan

Field	Detail
Project	BloodLink — Real-time Blood Bank Management & Intelligent Donor Matching
Version	2.0
Date	2026-05-01
Prepared By	QA Team
Status	Active

1. Introduction

1.1 Purpose

This document defines the comprehensive test plan for **BloodLink v2.0**, a full-stack clinical platform built with **NestJS 11** (backend), **Next.js 16** (frontend), and **PostgreSQL 17** (database). The system manages blood inventory across 8 ABO/Rh blood types, maintains a donor registry with eligibility tracking, processes emergency blood requests with 4 urgency levels, and runs an **Intelligent Matching Engine v2** that scores and ranks donors using a multi-factor algorithm incorporating reliability, Haversine-based geospatial proximity, exact-match bonuses, and WHO-compliant recency safety penalties.

1.2 Project References

Document	Location
README / System Specification	BloodLink/README.md
Class Diagram	BloodLink/class_diagram.md
Sequence Diagrams	BloodLink/sequence_diagrams.md
API Source (Backend)	BloodLink/backend/src/
UI Source (Frontend)	BloodLink/frontend/src/

1.3 Glossary

Term	Definition
Matching Engine v2	Multi-factor scoring algorithm: $\text{Score} = (\text{Reliability} \times W_r) + (\text{Proximity} \times W_p) + (\text{ExactMatch} \times W_e) - \text{RecencyPenalty}$
Haversine	Formula computing great-circle distance between two GPS coordinates
Recency Penalty	Safety deduction for donors who donated within the last 56 days (8 weeks, WHO guideline)

Term	Definition
Urgency-Adaptive Weights	Dynamic weight profiles (Wr, Wp, We) that change based on urgency: Critical, High, Medium, Low
Composite Score	Final donor ranking score clamped to 0–100

2. Test Objectives

#	Objective
O-1	Verify all 5 backend modules (Auth, Donors, Inventory, Emergencies, Notifications) expose correct REST API behavior
O-2	Validate the Matching Engine v2 produces mathematically correct composite scores for all urgency levels
O-3	Confirm ABO/Rh blood compatibility matrix returns only medically valid donor sets
O-4	Ensure Haversine distance calculations between all 15 Cairo districts are accurate to ± 0.5 km
O-5	Verify JWT authentication guards all protected endpoints and role-based access is enforced
O-6	Validate inventory threshold logic: Healthy (>80), Warning (31–80), Critical (≤ 30)
O-7	Confirm end-to-end emergency workflow: Create → Match → Notify → Resolve
O-8	Verify frontend UI renders correctly, toast notifications fire on all CRUD operations, and navigation guards redirect unauthenticated users
O-9	Ensure database seeder populates correct initial data (3 users, 8 donors, 8 inventory, 7 notifications, 3 emergencies)
O-10	Validate input validation via DTOs rejects malformed payloads with proper error messages

3. Scope

3.1 In Scope

Area	Details
Backend API	All 22 REST endpoints across Auth, Users, Donors, Inventory, Emergencies, Notifications
Matching Engine	Scoring formula, weight profiles, Haversine calc, recency penalty, tie-breaking logic
Blood Compatibility	Full 8×8 ABO/Rh compatibility matrix validation

Area	Details
Database	Entity schema (5 entities), seed data integrity, TypeORM transactions
Authentication	JWT sign/verify, bcrypt hashing, Passport strategy, token interceptor
Frontend Pages	Login, Dashboard, Inventory, Donors, Emergencies, Notifications, Profile (7 pages)
Frontend Components	Navbar (unread badge), Toast notification system
Input Validation	class-validator DTOs with whitelist/transform/forbidNonWhitelisted
CORS	Origin whitelist for localhost:3000 and localhost:3002

3.2 Out of Scope

- Load/stress testing beyond functional verification
- SMS/email gateway integration (notifications are in-app only)
- Mobile-native application testing
- Penetration testing / OWASP audit
- Cross-browser testing beyond Chrome and Firefox

4. Test Strategy

4.1 Test Levels

Level 1 — Unit Testing

Aspect	Detail
Target	Individual service methods, utility functions, entity validation
Framework	Jest (NestJS built-in)
Mocking	TypeORM repositories mocked via <code>jest.fn()</code> , JwtService mocked
Coverage Target	≥ 80% line coverage for service files

Key areas:

- `EmergenciesService` : `haversineKm()`, `getCityCoord()`, `daysSinceLastDonation()`, scoring formula
- `InventoryService` : `getStatus()` threshold logic
- `AuthService` : `login()` credential validation, `sanitize()` password exclusion
- `DonorsService` : `findCompatible()` blood type filtering
- `NotificationsService` : CRUD operations

Level 2 — Integration Testing

Aspect	Detail
Target	Module-to-module interactions, controller-to-service-to-database flows
Framework	Jest + Supertest + NestJS <code>Test.createTestingModule()</code>
Database	In-memory SQLite or dedicated test PostgreSQL schema

Key flows:

- Auth → JWT → Protected endpoint access
- Emergency creation → Matching Engine → Notification generation
- Inventory CRUD → status auto-calculation
- Donor toggle availability → reflected in matching results

Level 3 — System / End-to-End Testing

Aspect	Detail
Target	Full user workflows from browser to database
Framework	Cypress or Playwright
Coverage	All 7 frontend pages, all CRUD flows, toast notifications

Key scenarios:

- Complete login → dashboard → emergency → match → resolve workflow
- Donor CRUD with search, filter, and availability toggle
- Inventory management with threshold badge verification
- Notification mark-read, dismiss, bulk actions

Level 4 — Manual / Exploratory Testing

Aspect	Detail
Target	UI/UX quality, responsive layout, glassmorphism design, edge cases
Approach	Session-based exploratory testing
Focus	Visual consistency, animation smoothness, error state handling

4.2 Test Types

Type	Approach
Functional	Verify each API endpoint and UI action produces correct results
Negative	Invalid inputs, missing fields, expired tokens, non-existent IDs
Boundary	Inventory at 0, 30, 31, 80, 81 units; donor age limits; recency at 0, 56, 57 days

Type	Approach
Security	JWT expiry, missing token, tampered token, SQL injection via search params
Compatibility	Chrome 120+, Firefox 120+, Edge 120+
Regression	Re-run full suite after every feature branch merge

5. Test Environment

5.1 Hardware / Software

Component	Specification
OS	Windows 11 / Ubuntu 22.04
Node.js	≥ 18.x
npm	≥ 9.x
PostgreSQL	17.x (local or Docker)
Browser	Chrome 120+ (primary), Firefox 120+ (secondary)

5.2 Application Stack

Layer	Technology	Port
Backend	NestJS 11 + TypeORM 0.3	localhost:3001/api
Frontend	Next.js 16 + React 19 + Tailwind 4	localhost:3000
Database	PostgreSQL 17	localhost:5432

5.3 Test Data

The database seeder (`seed.service.ts`) provides deterministic test data:

Entity	Count	Key Details
Users	3	Admin (<code>admin@bloodlink.org</code>), Staff (<code>staff@qasr.org</code>), Manager (<code>manager@dar-el-fouad.org</code>)
Donors	8	All 8 blood types, 8 Cairo districts, reliability 75–99, mixed availability/eligibility
Inventory	8	One per blood type; O+ at 420 (Healthy), O- at 85 (Critical), AB- at 15 (Critical)
Notifications	7	5 types, 3 unread + 4 read, all assigned to admin user
Emergencies	3	Critical (O-, 6 units), High (A+, 4 units), Medium (AB-, 2 units)

5.4 Test Accounts

Role	Email	Password
Admin	admin@bloodlink.org	admin123
Staff	staff@qasr.org	staff123
Manager	manager@dar-elfouad.org	manager123

6. Module Breakdown & Feature Coverage

6.1 Auth Module (auth/)

Feature	Test Type	Priority
Login with valid credentials → returns JWT + sanitized user (no password)	Unit, Integration	● Critical
Login with wrong password → 401 UnauthorizedException	Unit, Negative	● Critical
Login with non-existent email → 401 UnauthorizedException	Unit, Negative	● Critical
GET /auth/me with valid token → returns current user profile	Integration	● Critical
GET /auth/me with expired/invalid token → 401	Integration, Security	● Critical
Password stored as bcrypt hash (not plaintext)	Unit	● Critical
JWT payload contains sub , email , role	Unit	● Medium

6.2 Donors Module (donors/)

Feature	Test Type	Priority
GET /donors → returns all donors sorted by name ASC	Integration	● High
GET /donors?search=Mohamed → filters by name/email/city/bloodType (ILIKE)	Integration	● High
GET /donors?bloodType=O+ → filters by exact blood type	Integration	● High
GET /donors?available=true → filters by availability	Integration	● High
POST /donors with valid DTO → creates donor with defaults (reliability=50, available=true)	Integration	● High
POST /donors with duplicate email → database unique constraint error	Negative	● Medium

Feature	Test Type	Priority
PATCH /donors/:id → partial update of any field	Integration	High
PATCH /donors/:id/toggle-availability → flips boolean	Unit, Integration	High
DELETE /donors/:id → removes donor, returns { deleted: true }	Integration	Medium
findCompatible('O+') → returns only O+, O- donors who are available AND eligible	Unit	Critical
findCompatible('AB+') → returns all 8 blood types (universal recipient)	Unit	Critical
Non-existent donor ID → 404 NotFoundException	Negative	Medium

6.3 Inventory Module (inventory/)

Feature	Test Type	Priority
GET /inventory → returns all items sorted by type ASC	Integration	High
GET /inventory?status=Critical → filters by status	Integration	Medium
POST /inventory → auto-calculates status and critical flag	Unit, Integration	Critical
getStatus(30) → Critical	Unit, Boundary	Critical
getStatus(31) → Warning	Unit, Boundary	Critical
getStatus(80) → Warning	Unit, Boundary	High
getStatus(81) → Healthy	Unit, Boundary	High
critical flag set to true when units ≤ 30	Unit	High
PATCH /inventory/:id with new units → recalculates status	Integration	High
DELETE /inventory/:id → removes item	Integration	Medium

6.4 Emergencies Module (emergencies/)

Feature	Test Type	Priority
GET /emergencies → returns only ACTIVE requests, ordered by createdAt DESC	Integration	High
POST /emergencies → creates with default status=Active, distance defaults to 0	Integration	High
PATCH /emergencies/:id → partial update of hospital, department, requiredType, unitsNeeded, urgency	Integration	Medium
DELETE /emergencies/:id → removes request	Integration	Medium

Feature	Test Type	Priority
<code>PATCH /emergencies/:id/resolve</code> → sets status to Resolved	Integration	High
<code>GET /emergencies/:id/match</code> → runs Matching Engine, returns scored donors	Integration	Critical
<code>POST /emergencies/:id/notify</code> → creates notification for requesting user	Integration	High

6.5 Matching Engine v2 (within `emergencies.service.ts`)

Feature	Test Type	Priority
Haversine: Nasr City → Heliopolis ≈ 4.5 km	Unit	Critical
Haversine: Same point → 0 km	Unit, Boundary	Medium
Proximity score: $\max(0, 100 - \text{distanceKm} \times 2.5)$ — 40 km+ → 0	Unit	Critical
getCityCoord: exact match 'nasr city' → correct coords	Unit	High
getCityCoord: fuzzy match 'Nasr City Hospital' → Nasr City coords	Unit	High
getCityCoord: unknown city → defaults to Downtown (30.0444, 31.2357)	Unit	High
Exact match bonus: donor O+ for request O+ → 100	Unit	Critical
Exact match bonus: donor O- for request O+ → 0 (compatible but not exact)	Unit	Critical
Recency penalty: 0 days since donation → penalty = 25	Unit, Boundary	Critical
Recency penalty: 28 days → penalty = 13 (rounded from 12.5)	Unit	High
Recency penalty: 56 days → penalty = 0	Unit, Boundary	Critical
Recency penalty: 60 days → penalty = 0 (no negative penalty)	Unit, Boundary	Critical
Recency penalty: null lastDonation → daysSince = 999 → penalty = 0	Unit	High
Weights (Critical): $W_r=0.35, W_p=0.45, W_e=0.20$	Unit	Critical
Weights (High): $W_r=0.45, W_p=0.35, W_e=0.20$	Unit	High
Weights (Medium): $W_r=0.55, W_p=0.25, W_e=0.20$	Unit	High
Weights (Low): $W_r=0.60, W_p=0.20, W_e=0.20$	Unit	High
Unknown urgency → falls back to Medium weights	Unit	Medium
Score clamping: rawScore negative → clamped to 0	Unit, Boundary	High

Feature	Test Type	Priority
Score clamping: rawScore > 100 → clamped to 100	Unit, Boundary	● High
Sort order: descending by score → exact match → closest distance	Unit	● Critical
Top-10 cap: returns at most 10 donors	Unit	● Medium
ETA calculation: <code>max(5, round(distanceKm × 3))</code> minutes	Unit	● Medium
Response shape: includes <code>algorithm</code> , <code>weights</code> , <code>totalCompatible</code> , <code>highReliability</code> , <code>exactMatches</code>	Integration	● Medium

6.6 Notifications Module (`notifications/`)

Feature	Test Type	Priority
<code>GET /notifications</code> → returns all for current user, ordered by timestamp DESC	Integration	● High
<code>PATCH /notifications/:id/read</code> → sets read = true	Integration	● Medium
<code>PATCH /notifications/read-all</code> → bulk update for user	Integration	● Medium
<code>DELETE /notifications/:id</code> → removes single notification	Integration	● Medium
<code>DELETE /notifications/clear-all</code> → removes all for user	Integration	● Medium
Non-existent notification ID → 404	Negative	● Medium

6.7 Frontend Pages

Page	Key Test Scenarios	Priority
Login (<code>/login</code>)	Valid login redirects to <code>/</code> , invalid shows error, token stored in localStorage	● Critical
Dashboard (<code>/</code>)	Loads stats, displays inventory summary, recent activity	● High
Inventory (<code>/inventory</code>)	CRUD operations, status badges (Healthy/Warning/Critical), filter by status	● High
Donors (<code>/donors</code>)	Search, blood type filter, availability filter, toggle availability, CRUD	● High
Emergencies (<code>/emergencies</code>)	Create request, run matching engine, view scored donors, notify, resolve	● Critical
Notifications (<code>/notifications</code>)	Mark read, dismiss, mark all read, clear all, unread badge in Navbar	● Medium
Profile (<code>/profile</code>)	View/edit user profile, hospital assignment	● Medium

6.8 Cross-Cutting Concerns

Feature	Test Type	Priority
JWT interceptor attaches token to all requests	Integration	● Critical
Unauthenticated access redirects to <code>/login</code>	E2E	● Critical
Toast notifications appear for success/error on all CRUD	E2E	● High
Navbar unread notification count badge updates in real-time	E2E	● Medium
CORS allows only localhost:3000 and localhost:3002	Integration	● Medium
ValidationPipe rejects unknown/extra fields (forbidNonWhitelisted)	Integration	● High

7. Test Schedule

Phase	Duration	Activities
Phase 1: Unit Tests	Week 1	Write and execute unit tests for all 5 service files + Matching Engine
Phase 2: Integration Tests	Week 2	API endpoint testing with Supertest, DB transaction verification
Phase 3: System/E2E Tests	Week 3	Full browser-based workflow testing with Cypress
Phase 4: Regression + UAT	Week 4	Re-run all suites, exploratory testing, stakeholder sign-off

8. Entry & Exit Criteria

8.1 Entry Criteria

- All source code committed and builds successfully (`npm run build` passes)
- Database schema synchronized and seed data loads without errors
- Backend starts on port 3001, frontend starts on port 3000
- Test environment configured with valid `.env` and PostgreSQL connection

8.2 Exit Criteria

- **Unit tests:** $\geq 80\%$ line coverage, 100% of critical-priority tests pass
- **Integration tests:** All 22 API endpoints tested with positive + negative cases
- **E2E tests:** All 7 pages exercised, all CRUD workflows verified
- **Zero** Critical or High severity defects remain open
- All Medium severity defects documented with workarounds

9. Risk Analysis

Risk	Likelihood	Impact	Mitigation
PostgreSQL connection failure in test env	Medium	High	Use Docker container with fixed port; fallback to SQLite for unit tests
Seed data changes break test assertions	Medium	Medium	Pin seed data values in test fixtures; don't depend on live seeder
Matching Engine floating-point precision	Low	High	Use <code>toFixed(1)</code> and <code>Math.round()</code> consistently; test with tolerance
JWT secret mismatch between environments	Low	High	Use dedicated <code>.env.test</code> with known secret
Frontend API base URL hardcoded to localhost	Medium	Low	Documented as known limitation; use env variable for staging
Recency penalty edge case at exactly 56 days	Medium	High	Explicit boundary tests at 55, 56, 57 days

10. Roles & Responsibilities

Role	Responsibility
QA Lead	Maintains this test plan, reviews test results, manages defect triage
Backend Developer	Writes unit tests for services, fixes backend defects
Frontend Developer	Writes E2E tests, fixes UI defects
Project Manager	Approves exit criteria, schedules UAT sessions

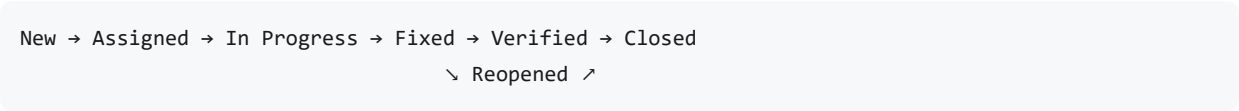
11. Defect Management

11.1 Severity Levels

Severity	Definition	Response Time
 Critical	System crash, data loss, security breach, matching engine produces wrong results	Fix within 24 hours
 High	Major feature broken (e.g., cannot create emergency, login fails)	Fix within 48 hours

Severity	Definition	Response Time
 Medium	Feature works with workaround (e.g., filter doesn't reset, minor UI glitch)	Fix within 1 week
 Low	Cosmetic issue, typo, minor UX inconsistency	Fix in next release

11.2 Defect Lifecycle



12. Test Deliverables

Deliverable	File
Test Plan (this document)	test_docs/Test_Plan.md
Test Cases (Unit, Integration, System)	test_docs/Test_Cases.md
Bug Report Template + Sample Bugs	test_docs/Bug_Reports.md
Test Evidence (Logs + Screenshots)	test_docs/Test_Evidence.md

13. Traceability Matrix

Requirement	Test Cases	Module
REQ-01: User authentication with JWT	UT-AUTH-01 to UT-AUTH-07, IT-AUTH-01 to IT-AUTH-03, E2E-001	Auth
REQ-02: Donor CRUD + search/filter	UT-DONOR-01 to UT-DONOR-04, IT-DONOR-01 to IT-DONOR-08, E2E-003	Donors
REQ-03: Blood compatibility matrix	UT-COMPAT-01 to UT-COMPAT-08	Donors
REQ-04: Inventory management + thresholds	UT-INV-01 to UT-INV-05, IT-INV-01 to IT-INV-05	Inventory
REQ-05: Emergency request lifecycle	IT-EMRG-01 to IT-EMRG-05, E2E-002	Emergencies
REQ-06: Matching Engine v2 scoring	UT-MATCH-01 to UT-MATCH-18	Emergencies
REQ-07: Haversine distance calculation	UT-HAV-01 to UT-HAV-03	Emergencies

Requirement	Test Cases	Module
REQ-08: Notification system	IT-NOTIF-01 to IT-NOTIF-05	Notifications
REQ-09: Input validation (DTOs)	IT-VAL-01 to IT-VAL-04	Cross-cutting
REQ-10: Role-based access control	IT-RBAC-01 to IT-RBAC-03	Auth

14. Approval

Name	Role	Signature	Date
	QA Lead		
	Project Manager		
	Tech Lead		